RELATION EXTRACTION WITH GRAPH NEURAL NETWORKS IN SEMI STRUCTURED DOCUMENTS

A PROJECT REPORT

Submitted by

KESA VARSHITHA [CB.EN.U4CSE19121]
N D N JAHNAVI [CB.EN.U4CSE19135]
R. SNEHA NAIR [CB.EN.U4CSE19139]
SITHARA J P [CB.EN.U4CSE19148]

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

AMRITA SCHOOL OF COMPUTING

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE - 641 112

JUNE 2023

AMRITA VISHWA VIDYAPEETHAM

AMRITA SCHOOL OF COMPUTING, COIMBATORE – 641 112

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **RELATION EXTRACTION WITH GRAPH NEURAL NETWORKS IN SEMI STRUCTURED DOCUMENTS** submitted by Kesa Varshitha (CB.EN.U4CSE19121), N D N Jahnavi (CB.EN.U4CSE19135), R Sneha Nair (CB.EN.U4CSE19139), Sithara J P (CB.EN.U4CSE19148) in partial fulfillment of the requirements for the award of Degree **Bachelor of Technology** in Computer Science and Engineering is a bonafide record of the work carried out under our guidance and supervision at the Department of Computer Science and Engineering, Amrita School of Computing, Coimbatore.

Ms.BINDU K.R.
Assistant professor
Department of CSE

Dr.Vidhya Balasubramanian
Chairperson
Department of CSE

Evaluated on:

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We, the undersigned solemnly declare that the project report **RELATION EXTRACTION WITH GRAPH NEURAL NETWORKS IN SEMI STRUCTURED DOCUMENTS** is based on our own work carried out during the course of our study under the supervision of Ms.BINDU K.R., Assistant professor, Computer Science and Engineering, and has not formed the basis for the award of any other degree or diploma, in this or any other Institution or University. In keeping with the ethical practice in reporting scientific information, due acknowledgement has been made wherever the findings of others has been cited.

Kesa Varshitha [CB.EN.U4CSE19121] -

N D N Jahnavi [CB.EN.U4CSE19135] -

R. Sneha Nair [CB.EN.U4CSE19139] - 

Sithara J P [CB.EN.U4CSE19148] -

ABSTRACT

Relation extraction is a task in natural language processing Natural Language Processing (NLP) that involves identifying and classifying the semantic relationships between entities in text. Entity-relationship extraction is a challenging task due to the complexity and ambiguity of natural language. The current state-of-the-art relation extraction models for Indian language rely on hand-crafted features which limit their performance and scalability. The objective of this project is to develop a novel approach for relation extraction on Hindi language text using Graph Neural Networks Graph Neural Networks (GNNs).

We first construct a graph representation of the input text in Hindi, where each named entity is represented as a node and the edges between nodes capture the syntactic and semantic relationships between them. We then train a Graph Neural Network (GNN) to perform Named Entity Relation Named Entity Recognition (NER) and relation extraction by encoding the graph structure and node features into a low-dimensional embedding space. Our model leverages the inherent connections between NER and relation extraction tasks and allows them to be jointly optimized, resulting in improved performance compared to standalone models. Our model can identify both binary and n-ary relations, and it can handle complex entity types.

We evaluate our model on an available Hindi language entity-relation extraction dataset and achieve state-of-the-art performance compared to existing methods. Our model would demonstrate consistent performance improvements, without requiring heavy feature engineering nor additional language-specific knowledge.

Our approach has the potential to be applied to other languages where similar challenges exist, such as other Indic languages with complex morphological and syntactic structures. The project can be extended to focus on relation extraction in specific domains, such as biomedical, financial or legal texts.

ACKNOWLEDGEMENT

We would like to express our deep gratitude to our beloved Satguru **Sri Mata Amritanandamayi Devi** for providing the bright academic climate at this university, which has made this entire task appreciable. This acknowledgment is intended to thank all those people involved directly or indirectly with our project. We would like to thank our Pro Chancellor **Swami Abhayamritananda Puri**, Vice Chancellor **Dr.Venkat Rangan.P** and **Dr.Bharat Jayaraman**, Dean, Faculty of AI & Computing(Computing), Amrita Vishwa Vidyapeetham for providing us with the necessary infrastructure required for the completion of the project. We express our thanks to **Dr.Vidhya Balasubramanian**, Chairperson, Department of Computer Science Engineering and Principal, School of Computing, Amrita Vishwa Vidyapeetham, **Dr.C.Shunmuga Velayutham and Dr.N.Lalithamani,** Vice Chairpersons of the Department of Computer Science and Engineering for their valuable help and support during our study. We express our gratitude to our guide, **Ms.Bindu K.R.** for their guidance, support and supervision. We feel extremely grateful to **Dr. Swapna T. R., Ms. Manjusha R., Ms. Nalinadevi** for their feedback and encouragement which helped us to complete the project. We would also like to thank the entire fraternity of the Department of Computer Science and Engineering. We would like to extend our sincere thanks to our family and friends for helping and motivating us during the course of the project. Finally, we would like to thank all those who have helped, guided and encouraged us directly or indirectly during the project work. Last but not the least, we thank God for his blessings which made our project a success.

Kesa Varshitha [CB.EN.U4CSE19121]

N D N Jahnavi [CB.EN.U4CSE19135]

R Sneha Nair [CB.EN.U4CSE19139]

Sithara J P [CB.EN.U4CSE19148]

TABLE OF CONTENTS

LIST OF FIGURES

ABBREVIATIONS

Bi-LSTM	Bidirectional Long Short-Term Memory
BIO	Begin, Inside, Outside
CRF	Conditional Random Field
F1	F-Measure or F-Score
GCNs	Graph Convolutional Networks
GNNs	Graph Neural Networks
GPGNN	Generated parameter Graph Neural Network
NER	Named Entity Recognition
NLP	Natural Language Processing
PoS	Parts of Speech Tagging

List of Symbols

α, β	Damping constants
θ	Angle of twist, rad
ω	Angular velocity, rad/s
b	Width of the beam, m
h	Height of the beam, m
$\{f(t)\}$	force vector
$[K^e]$	Element stiffness matrix
$[M^e]$	Element mass matrix
$\{q(t)\}$	Displacement vector
$\{\dot{q}(t)\}$	Velocity vector
$\{\ddot{q}(t)\}$	Acceleration vector

Chapter 1

INTRODUCTION

An organization may implement AI-driven decision-making processes to enhance their efficiency by in- corporating heterogeneous documents. To automate these transactions successfully, we must be able to incorporate semantic understanding, in addition to the traditional Optical Character Recognitions (OCR). Information extraction (IE) is the process of automatically retrieving structured data from semi-structured and scanned machine-readable documents. This requires named entity recognition (NER) and the discov- ery of relationships between document terms. NER identifies word or phrase spans in unstructured text (the entity) and classify them as belonging to a particular class (the entity type). Relation Extraction (RE) identifies relationships implied in the text between a pair of entities. NER and relation extraction (RE) are two important tasks in information retrieval.

Identification and extraction of the semantic relationships between entities in unstructured text data is a crucial step in the natural language processing (NLP) process known as relational extraction. Events, affiliations, dependencies, and more expressions of these relationships are possible. Recognising and comprehending the underlying relationships between elements in a text document is the aim of relational extraction. In text data, relationships between things can be intricate and varied. Rule-based or statistical models, which are traditional relational extraction methods, are constrained in their ability to handle complicated and varied text data.

The activity of recognising and classifying named things in unstructured text data is known as named entity recognition (NER), and it is a critical component of natural language processing (NLP). These entities can be individuals, businesses, places, and more. The process of "relational extraction," on the other hand, entails locating and obtaining the semantic connections between items in text data.

Recent developments in machine learning, particularly deep learning, offer the potential to enhance the efficiency of relationship extraction and NER tasks. A promising method to handle these tasks has emerged: graph neural networks (GNNs), a class of deep learning model that can accurately represent the intricate relationships between items in a text document.

Using deep learning models, relational extraction entails encoding text data as numerical representations, such as word embeddings. The models are taught to identify and extract from the text relationships between entities. Depending on the available labelled training data, they can be supervised, unsupervised, or semi-supervised.

Deep learning models called graph neural networks (GNNs) have showed promise in overcoming these restrictions and enhancing relational extraction task performance. These neural network architectures function with data that is organised into graphs. The entities and relationships in text data can be represented as nodes and edges in a graph, respectively, in the context of NLP. In order to extract significant relationships between entities, GNNs are able to efficiently capture the dependencies and relationships between nodes and propagate information between them.

Relational extraction uses GNNs to graph-encode the entities and relationships in a text document and train a model to understand how they are related. By passing messages depending on the representations of their neighbours, each layer of the model changes the node representations. The model consists of several layers of neural networks that use the graph data as input.

Since recognising the entities is a requirement for discovering their relationships, the interaction between NER and relational extraction is crucial. Furthermore, enhancing the overall efficiency of the relational extraction work requires correct named entity recognition.

GNNs can be employed in the context of NER and relational extraction to graph-encode the text data and learn to extract significant relationships between entities. Accurate

NER and relational extraction are made possible by the GNN model's ability to record dependencies between entities and propagate information between them. In order to accurately represent complicated entity interactions, GNN-based models excel in integrating a variety of data, including word embeddings, entity types, and context. They improve overall performance by utilising attention processes to draw attention to crucial graph links.

Overall, the use of GNNs in relational extraction and NER has produced encouraging outcomes in a number of applications, including sentiment analysis, question-answering, and information retrieval. GNNs and other deep learning models are probably going to become more crucial as the field develops in terms of enhancing our capacity to extract valuable information from unstructured text input.

1.1 Problem Definition

To implement relation extraction using Graph neural networks

Using graph neural networks (GNNs), the relational extraction challenge entails extracting relationships and structured information from unstructured text data. The objective is to automatically recognise, extract, and represent important entities, properties, and relationships from textual sources in the form of a structured graph.

The task, given a set of documents, entails:

Entity Extraction : identifying and extracting relevant entities (e.g., persons, organizations, locations) from the text.

Relation Extraction : figuring out the connections between the many entities mentioned in the text. This entails determining the various relationship kinds (such as "works for" and "located in") as well as the connected entities involved in each relationship.

Graph Representation : generating a representation of a structured graph that includes the extracted entities and their connections. The information at the entity level as well as the relationships between entities should be represented in the graph.

Knowledge Inference : higher-level reasoning tasks, such as predicting missing relationships, assuming new information, or responding to inquiries based on the extracted data, can be carried out using the graph model.

Natural language processing, information extraction, and graph neural network approaches must all be used to solve the difficult problem of relationship extraction with graph neural networks. Information retrieval, question-answering systems, knowledge base creation, and recommendation systems are just a few of the many fields in which it has extensive applications.

The goal is to create efficient algorithms and models that can handle the complexity of unstructured text input, extract significant relationships, and represent them in a structured graph format to facilitate effective knowledge representation and inference.

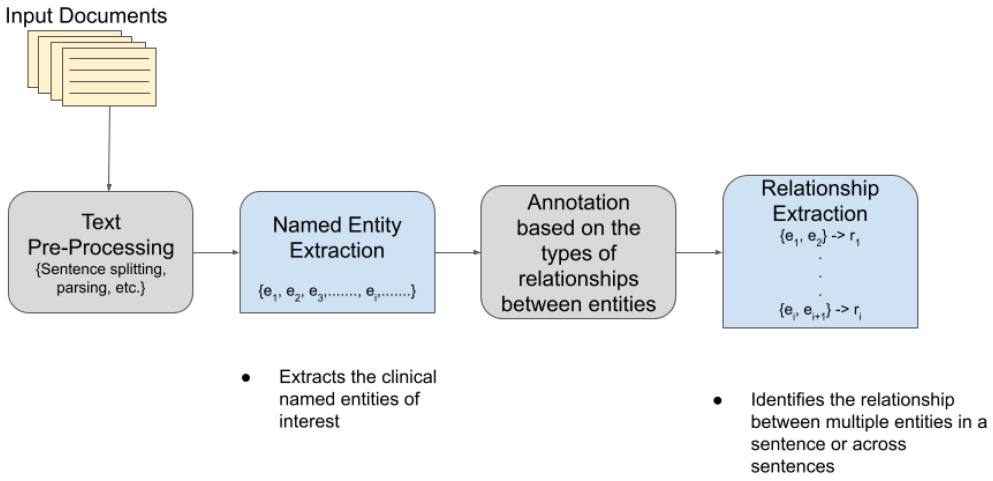


Figure 1.1: Relation Extraction

Chapter 2

LITERATURE SURVEY

2.1 Graph Neural Networks with Generated Parameters for Relation Extraction

The Study by Shao et al. (2021b) In the research paper "Graph Neural Networks with Generated Parameters for Relation Extraction" (2019) introduces a novel approach to relation extraction using graph neural networks (GNNs) with dynamically generated parameters. The word embeddings required for this model are taken from Zeng et al. (2014).By leveraging GNNs and generating relation-specific parameters, the authors aim to overcome limitations in traditional methods. The proposed model represents text as a graph, allowing for the capture of contextual information and entity relationships. The GNNs propagate information through the graph, enabling effective feature representation for relation extraction. Experimental results demonstrate that the proposed approach outperforms existing methods, achieving state-of-the-art performance on relation extraction benchmarks. This research highlights the potential of GNNs with generated parameters for advancing relation extraction tasks in natural language processing (NLP), offering insights for improving information extraction.

2.2 Named Entity Recognition and Relation Extraction with Graph Neural Networks in Semi Structured Documents

The Study By Carbonell et al. (2021) proposes a novel method for named entity recognition and relation prediction in historical Chinese documents using Graph Neural Networks (GNNs).This paper emphasizes relational reasoning in the natural language domain, showcasing the capability of neural networks to capture pair-wise relationships between entities, as demonstrated in previous works Shao et al. (2021a). His approach

involves detecting semantic groups of words, classifying entities into predefined categories, and uncovering meaningful pairwise relationships between them. By representing the document as a graph and leveraging the k-nearest neighbors (k-NN) approach, the system accurately predicts word links. The GNN-based method outperforms the BERT approach in entity labeling by capturing contextual relations between words. It achieves this by effectively clustering elements and evaluating similarities using the Adjusted Rand Index (ARI). The system demonstrates its ability to detect entities, classify them, and uncover relationships, presenting a promising approach for analyzing historical Chinese documents.

2.3 Graph Convolutional Networks for Named Entity Recognition

The study by Cetoli et al. (2017) addresses the application of Graph Convolutional Networks (GCNs) to Named Entity Recognition (NER).The paper emphasizes the positive impact of sentence grammar, particularly dependency trees, on improving NER accuracy. It highlights the significance of linguistic features, such as dependency trees, in enhancing NER performance.

In this paper the authors propose a novel approach that leverages the graph structure of input text to enhance NER systems. It introduces a specialized architecture for GCNs in NER, employing graph convolutional layers to capture contextual information from neighbouring words and attention mechanisms to focus on relevant aspects of the input graph. Additionally, a unique labelling scheme is introduced, utilizing the graph structure to assign labels to named entities. The paper compares the proposed GCN-based approach with traditional sequence-based models like BILSTM-CRF using benchmark datasets. The experiments demonstrate that Graph Convolutional Networks for Named Entity Recognition outperform conventional models and achieve state-of-the-art performance on multiple NER datasets. Combining GCNs with Part-of-Speech (POS) tag embeddings yields a significant improvement of 4.6±0.6 per in the F1 score.

The research paper suggests promising directions for future research, such as exploring additional techniques in conjunction with GCNs, including gating input vectors or neighbouring word prediction, to further advance NER performance. Overall, the paper presents a compelling approach for utilizing GCNs in Named Entity Recognition and contributes to the ongoing progress in this field. Wu et al. (2020) provides a comprehensive survey of Graph Neural Networks (GNNs) by exploring their architectures, training techniques, and applications. It offers a thorough overview of the advancements and challenges in the field, serving as a valuable resource for researchers and practitioners interested in GNNs.

2.4 Summary

In future research, there is potential for further enhancement of Graph Convolutional Networks (GCNs) by integrating various techniques. For instance, exploring the utilization of gating mechanisms to control input vector components or investigating neighboring word prediction in conjunction with Graph Convolutional Networks (GCNs) can complement their capabilities, thus improving the performance of relation extraction tasks. Notably, by incorporating multi-hop relational reasoning, the GP-GNN model exhibits significant improvements in relation extraction performance, demonstrating the importance of considering comprehensive connections between entities.

The GP-GNN framework presents a versatile approach that extends beyond textual data, enabling its application to graph generation problems involving unstructured inputs like images. This expands the potential scope of GP-GNN and establishes it as a generic framework suitable for diverse graph-related tasks. Additionally, as non-Indian languages dominate most NLP models, there is a need for experiments focusing on Indian languages, particularly Hindi. Such investigations would not only address the imbalance in language representation but also contribute valuable insights and advancements in NLP techniques tailored specifically to Indian languages.

2.5 DataSet

In Our study, the NYT29 dataset, a widely used dataset for Named Entity Recognition tasks, is utilized. The dataset comprises of text documents sourced from the New York Times and is annotated with named entities, including persons, organizations, locations,etc along with their respective labels.

Subsequently, the dataset is divided into three sets: training, validation, and test sets. The training set is employed to train the model, allowing it to learn from the provided data. The validation set serves the purpose of tuning hyperparameters and monitoring the model's performance during the training process. Finally, the test set is used to evaluate the final model's performance on unseen data, providing an unbiased assessment of its effectiveness.

2.6 Software/Tools Requirements

In this project, we utilized various software tools to facilitate different aspects of our research. Flask, a popular web development framework, was employed for web development tasks.

For relation extraction, we utilized Spacy, a powerful NLP library. Spacy provided efficient and accurate methods for extracting relationships between entities from textual data. Its robust functionality allowed us to perform advanced relation extraction tasks effectively.Additionally, Stanford NLP, a widely-used suite of natural language processing tools, was employed for part-of-speech tagging.The tool helped us to accurately assign grammatical tags to words in text, enabling us to gain insights into the syntactic structure of sentences.

In summary, Flask served as the web development framework, Spacy facilitated relation extraction, and Stanford NLP supported part-of-speech tagging in our research project. These software tools played vital roles in enabling various functionalities and enhancing the overall effectiveness of our project.

Chapter 3

PROPOSED SYSTEM

3.1 System Analysis

The proposed model introduces a novel approach to adaptively generate parameters for each input sentence, enabling it to effectively handle diverse sentence structures. By utilizing a transformer model and dependency parser, the system aims to extract named entities and establish context-aware relationships from unstructured text.

3.1.1 System requirement analysis

GP-GNN (GENERATED PARAMETERS FOR GRAPH NEURAL NETWORK)

GP-GNN, short for Generated Parameters for Graph Neural Network, is a specialized graph neural network model designed to augment the meaning of sentences by leveraging the structural information encoded in graphs. Its objective is to capture the relationships between words within a sentence and incorporate them into a more comprehensive representation that effectively captures semantic and contextual information. Here is an overview of how GP-GNN operates to enhance the meaningfulness of sentences:

Graph Construction

The representation of sentences in GP-GNN involves creating a graph where words serve as nodes and the connections between them are represented as edges. To construct this graph, dependency parsing algorithms such as the Stanford Dependency Parser or SpaCy's dependency parser can be employed to analyze the sentence's syntactic structure and generate the corresponding dependency graph.

Graph Neural Networks (GNNs)

GNNs are specialized neural network architectures developed specifically for handling data with graph structures. These networks operate on the nodes and edges of a graph,

continuously updating their representations by incorporating information from neighboring nodes. To facilitate this process, GNNs typically employ message passing algorithms, enabling nodes to exchange information iteratively through message passing iterations. This mechanism allows nodes to aggregate and refine information obtained from their neighboring nodes, leading to improved representations within the network.

GP-GNN Architecture

GP-GNN enhances the meaning of sentences by combining Graph Neural Networks (GNNs) with the concept of graph properties. Graph properties capture the overall characteristics or properties of a graph. In the context of sentences, these properties can capture syntactic or semantic features such as specific dependency patterns, average dependency path lengths, or the overall coherence of the graph structure. GP-GNN incorporates graph properties as extra input to the GNN, allowing the model to learn and prioritize important structural features during the message passing process. This integration enhances the model's ability to capture the significance of sentences.

Message Passing with Graph Properties

In GP-GNN, each word node in the sentence graph is initially assigned a representation. During each iteration of message passing, the nodes communicate with their neighboring nodes and update their representations based on the information they receive. In addition to the messages from neighbors, GP-GNN also incorporates graph properties as inputs during the message passing process. These graph properties serve as guidance for the model, enabling it to focus on relevant structural features and enhance the meaning of the sentence. The messages from neighbors and the graph properties are combined using learnable aggregation functions such as sum, mean, or attention mechanisms, allowing the node representations to be updated accordingly.

Sentence Representation and Meaning Enhancement

Once a fixed number of message passing iterations have been completed, the node representations in GP-GNN reach a stable state, resulting in a final representation for each word. These final representations encapsulate enhanced semantic and contextual information, having undergone refinement through message passing that considers both the

local interactions with neighboring words and the global graph properties. This resultant sentence representation holds significant value and can be leveraged for various downstream tasks like sentiment analysis, text classification, or information extraction, where accurately capturing the meaningfulness of the sentence is of crucial importance.

3.1.2 Module details of the system

Dataset preparation

Collect and preprocess training data: The system utilizes the widely used NYT29 dataset (New York Times), consisting of annotated text documents from the New York Times. These documents contain named entities and their corresponding labels. Split the dataset: The dataset is divided into training, validation, and test sets. The training set is used to train the model, the validation set assists in hyperparameter tuning and performance monitoring, and the test set evaluates the final model.

Model Architecture

Selection of prebuilt transformer model: The system strengthens prebuilt transformer models like en core web trf from SpaCy. These models are trained on large corpora, such as the Common Crawl, and employ deep learning architectures like BERT or GPT to learn contextual word representations. Fine-tuning the transformer model: The selected prebuilt transformer model is fine-tuned on the NER task using the annotated NYT29 dataset. Fine-tuning involves updating the model's parameters to make it more specific to the NER task and the target domain.

Training

Model initialization: An instance of the transformer-based NER model in SpaCy is created. Training loop definition: The training data is processed in batches within a loop. In each iteration, the model predicts named entities in the input text and calculates the loss. Model update: The loss is backpropagated through the model, and its parameters are updated using optimization algorithms like stochastic gradient descent (SGD) or Adam. Iterative training: The loop continues iterating over the training data for multiple epochs, allowing the model's parameters to be adjusted and the loss to be

minimized, thereby improving the model's performance on the NER task.

Evaluation and Tuning

Model validation: The model's performance is periodically evaluated on the validation set. Precision, recall, and F1-score metrics are calculated to assess the model's ability to correctly identify named entities. Hyperparameter tuning: Key hyperparameters such as learning rate, batch size, number of layers, or hidden units are adjusted to find the optimal configuration that maximizes the model's performance on the validation set.

Testing and Deployment

Final model evaluation: Once model training is complete, its performance is assessed on the test set to obtain a final evaluation of its effectiveness in recognizing named entities. Deployment: The trained NER model is integrated into the application or pipeline, enabling it to process new, unseen text and extract named entities effectively.

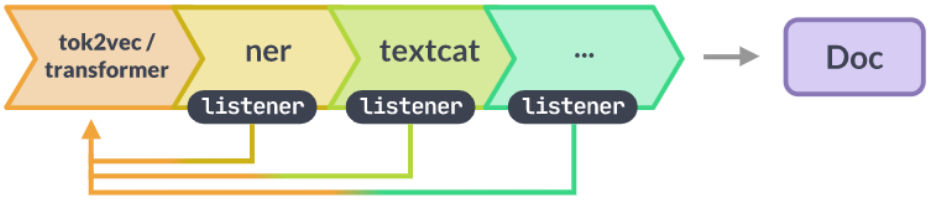
Dependency Parsing

Dependency parsing is utilized to identify the syntactic relationships between words in a sentence and represent them as a dependency tree. The tree consists of nodes representing words and directed edges denoting dependencies between the words. Popular dependency parsing algorithms such as Stanford Dependency Parser or SpaCy's built-in parser are employed to analyze the grammatical structure of the sentence and assign specific dependency labels to the edges in the tree.

Dependency-Based Relation Extraction

Once the named entities are recognized, the system utilizes the dependency tree to extract relationships between them. The following steps are involved: Identifying entity nodes: The nodes in the dependency tree corresponding to the recognized entities are located. Traversing the tree: Starting from the entity nodes, the system traverses the dependency tree to find the shortest path connecting the entities. This path represents the sequence of words and dependency labels linking the entities. Relation extraction: Patterns or heuristics are applied to the path to extract relevant relations between the entities. These patterns can encompass specific syntactic structures, dependency labels,

or contextual information surrounding the entities. Classification: Once the relations are extracted, they can be categorized into predefined classes or labeled based on the specific domain or task requirements.



Relation Tuple Extraction

To extract relation tuples, we use a pointer network approach. First, we combine the hidden representation of the current tuple with the encoder's hidden vectors and pass them through a bidirectional LSTM layer. This layer is followed by feed-forward networks (FFNs) that convert the hidden vectors into probabilities representing the start and end locations of the entities. These probabilities are obtained using softmax activation.

The first pointer network uses the bidirectional LSTM layer and FFNs to identify the start and end positions of the first entity. For the second entity, we concatenate various hidden vectors and pass them through a second pointer network. This process generates probabilities for the start and end positions of the second entity.

The relation embedding vector is determined using the argmax of the relation predictions. During training, the gold label relation embedding is used, ensuring the backpropagation process is unaffected. The decoder's sequence generation terminates when the predicted relation is EOS, which serves as the classification network.

During inference, we select the start and end positions of the entities to maximize the product of the four-pointer probabilities. Constraints are applied to avoid overlap and ensure valid entity positions. We determine the positions of Entity 1 first, followed by Entity 2, and vice versa. The pair of entities with the higher product of probabilities is

chosen. This decoding approach is known as PtrNetDecoding (PNDec).

Parameter Settings

To initialize the embeddings, we employ the Word2Vec tool on the NYT corpus (Mikolov et al., 2013). Random initialization is utilized for the character embeddings and relation embeddings. Throughout the training process, all embeddings are updated. The dimensions of the word embeddings (dw), relation embeddings (dr), character embeddings (dc), and character-based word feature (df) are set to 300, 300, 50, and 50, respectively. To extract the character-based word feature vector, a CNN filter width of 3 is employed, and the maximum word length is limited to 10. The decoder LSTM cell has a hidden dimension of 300, while the forward and backward LSTMs of the encoder have a hidden dimension of 150 each. The hidden dimension for the forward and backward LSTMs of the pointer networks is set to dp = 300. During training, a mini-batch size of 32 is used, and the network parameters are optimized using Adam (Kingma and Ba, 2015). Dropout layers with a fixed dropout rate of 0.3 are incorporated into the network architecture to mitigate overfitting.

Baselines and Evaluation Metrics

We evaluate our model against several state-of-the-art joint entities and relation extraction models. The models we compare against are as follows:

1. SPTree (Miwa and Bansal, 2016): This model combines sequence LSTM and Tree LSTM to perform end-to-end neural entity and relation extraction. It initially identifies entities using sequence LSTM and then employs Tree LSTM to determine relations between entity pairs.

2. Tagging (Zheng et al., 2017): This model is a neural sequence tagging approach that jointly extracts entities and relations. It utilizes an LSTM encoder and decoder, encoding entity and relation information through the Cartesian product of entity tags and relation tags. However, it struggles with cases involving overlapping entities.

3. CopyR (Zeng et al., 2018): CopyR adopts an encoder-decoder framework for joint

entity and relation extraction. It selectively copies the last token of an entity from the source sentence. Their top-performing multi-decoder model incorporates a fixed number of decoders, with each decoder responsible for extracting one tuple.

4. HRL (Takanobu et al., 2019): HRL employs a hierarchical reinforcement learning (RL) algorithm for tuple extraction. It utilizes high-level RL to identify relations and low-level RL, following a sequence tagging approach, to identify the two entities. However, the sequence tagging approach does not guarantee the extraction of precisely two entities in all scenarios.

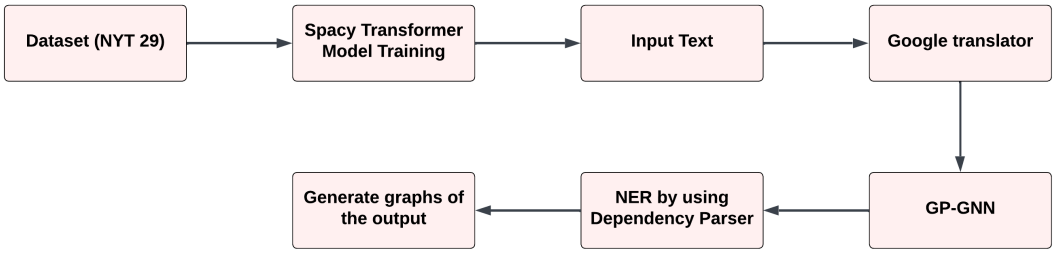
5. GraphR (Fu, Li, and Ma, 2019): This model treats each token in a sentence as a node in a graph, where edges represent relations. Graph convolution network (GCN) is utilized to predict relations for each edge, followed by filtering out certain relations.

6. N-gram Attention (Trisedya et al., 2019): This model employs an encoder-decoder approach with an N-gram attention mechanism for knowledge-base completion using distantly supervised data.

By conducting a comparison with these existing approaches, our objective is to demonstrate the effectiveness and superiority of our model in joint entity and relation extraction tasks.

3.2 System Design

3.2.1 Flow diagram of the system



3.2.2 Architecture

Encoder-Decoder Architecture

The encoder-decoder architecture employs special tokens to separate tuple components and distinguish overlapping entities. During inference, relation tuples can be easily extracted using these tokens. This uniform representation scheme requires a shared vocabulary encompassing entity tokens, relation tokens, and special tokens.

To generate relation tokens, the input sentence's important words indicating relations are utilized. Special tokens are used to differentiate the start of a relation tuple and the beginning of a tuple component. The encoder-decoder model effectively extracts relation tuples by generating entity tokens, identifying relation clue words, mapping them to relation tokens, and generating special tokens. Experiments confirm the effectiveness of the encoder-decoder approach.

Our model follows an encoder-decoder architecture, similar to machine translation models, where the decoder generates one word at a time. This allows for tuple extraction at the end of sequence generation due to the distinctive representation employed.

Pointer Network-Based Decoder

The model excludes special tokens and relation names from the word vocabulary and uses word embeddings only in the encoder along with character embeddings. It incorporates an additional relation embedding matrix in the decoder. Relation tuples are represented as a sequence, with each tuple consisting of start and end positions of entities and the connecting relation. The decoder includes an LSTM, two pointer networks for entity location, and a classification network for relation determination. During each time step, the decoder takes the source sentence encoding and the representation of previously generated tuples as input. It generates the hidden representation of the current tuple using an attention mechanism. To prevent duplicate tuples, information about all previously generated tuples is provided at each decoding step.

Word-level Decoder

In the word-level decoder, the target sequence is represented by word embedding vectors. The decoder generates one token at a time using an LSTM. The decoder takes the source sentence encoding and the previous target word embedding as input and generates the hidden representation of the current token. The decoder output is projected to the vocabulary using a linear layer. During training, the gold label target tokens are used directly. During inference, a masking technique is applied to the softmax operation at the projection layer. This masks out words that are not present in the current sentence, set of relations, or special tokens, ensuring that only entities from the source sentence are copied. If the decoder predicts a UNK token, it is replaced with the source word with the highest attention score. After decoding, tuples are extracted based on special tokens and duplicate and invalid tuples are removed.

WordDecoding outperforms other decoding approaches by achieving the highest F1 scores because of better error handling.

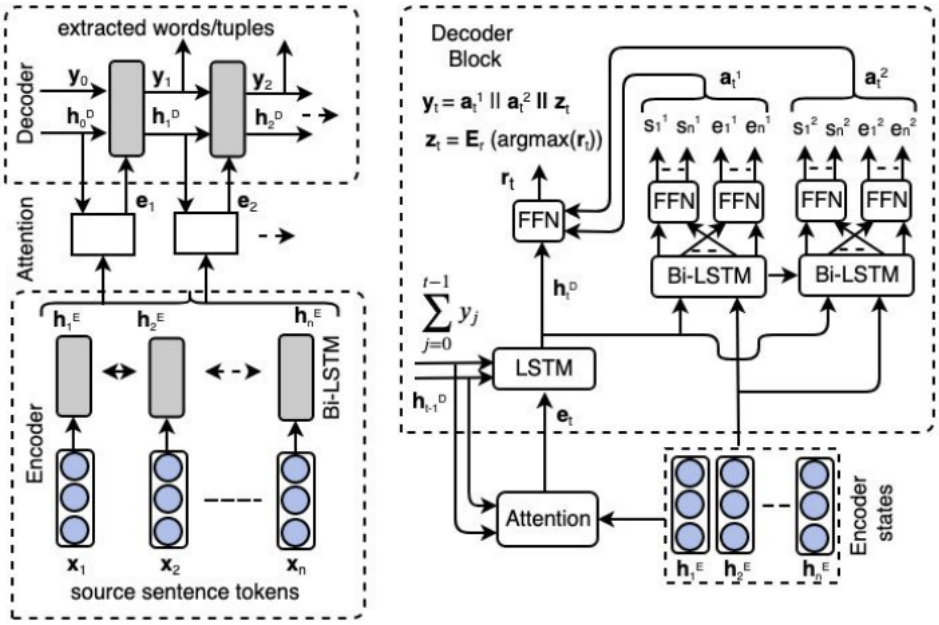


Figure 3.1: Encoder-decoder model (left) Pointer network-based decoder(right).

Chapter 4

IMPLEMENTATION AND TESTING

The steps involved in utilizing Graph Neural Networks (GNNs) with Generated Parameters for relation extraction are:

Data Preparation : Prepare the dataset for relation extraction, which includes labeled examples with entities, relations, and Hindi language text. Convert the Hindi text into numerical representations, such as word embeddings or subword embeddings, to create input features for the GNN model.

Graph Construction : Build a graph representation of the data, where entities are nodes and relations are edges connecting the entities. Assign appropriate node features to represent the entities and edge features to capture the relation information.

Model Architecture : Design the GNN model with Generated Parameters for relation extraction. This involves defining the layers, connectivity, and parameters of the GNN model specifically for Hindi text. Choose an appropriate GNN layer, such as Graph Convolutional Networks (GCN) or Graph Attention Networks (GAT), that can handle the graph structure and capture the contextual information in the Hindi text.

Parameter Generation : Incorporate a mechanism to generate relation-specific parameters. This could involve utilizing additional neural networks or techniques to dynamically generate parameters based on the input relations. These generated parameters adapt the GNN model to different relation types and enhance its ability to extract relations accurately from the Hindi text.

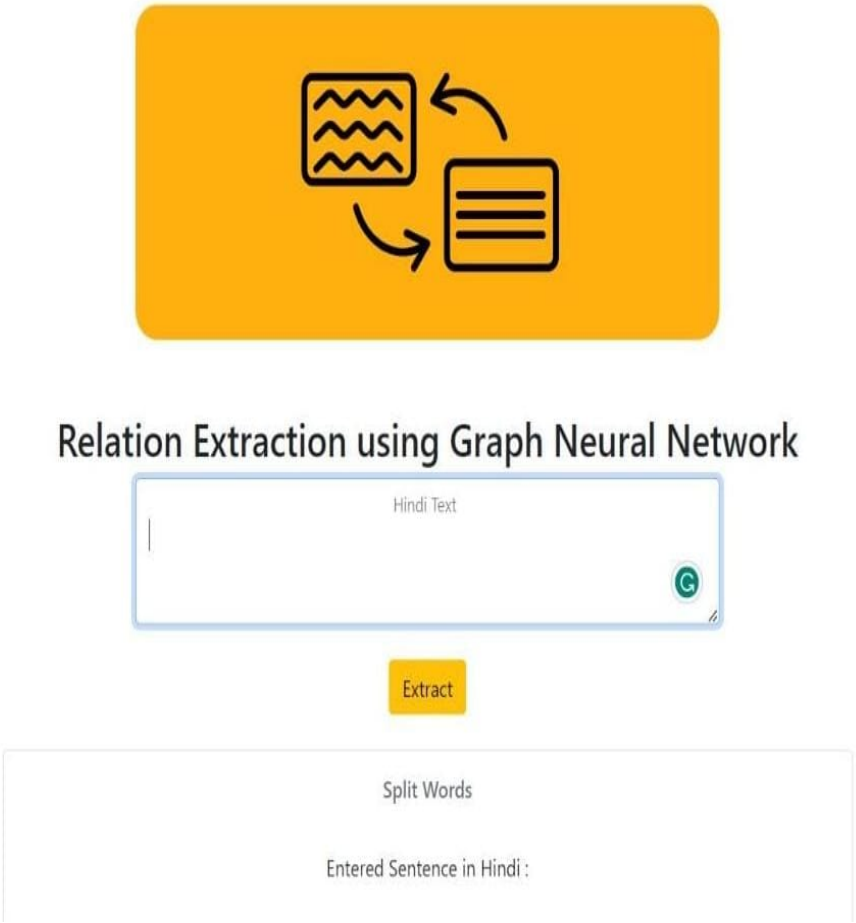
Training : Train the GNN model with Generated Parameters using the labeled dataset. Define a suitable loss function, such as cross-entropy loss, to optimize the model's performance. Utilize backpropagation and gradient-based optimization algorithms, such

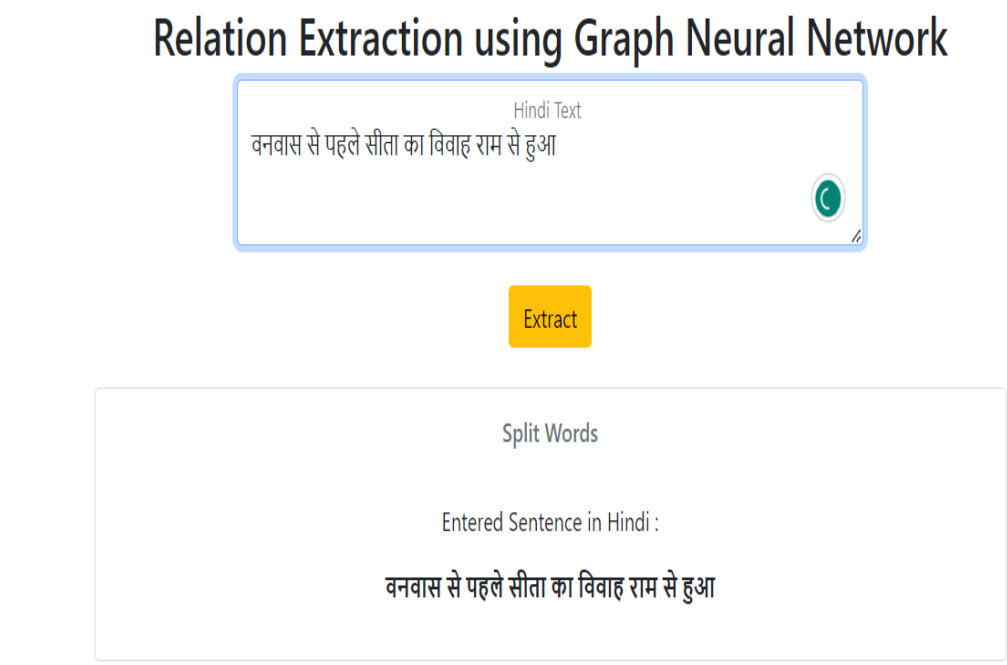
as stochastic gradient descent (SGD) or Adam, to update the model's parameters and minimize the loss.

Evaluation : Evaluate the trained model's performance on a separate validation or test dataset. Measure metrics such as accuracy, precision, recall, and F1-score to assess the model's ability to correctly predict relations between entities in the Hindi text.

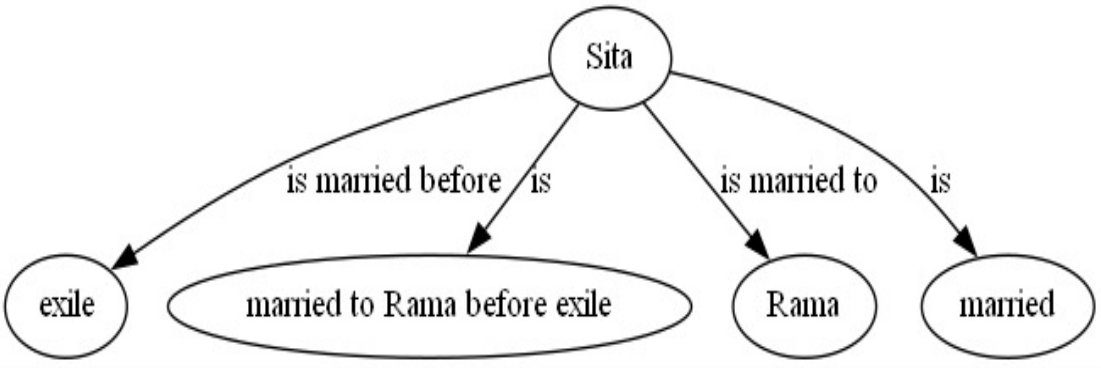
Inference : Apply the trained GNN model to perform relation extraction on new, unseen Hindi text. Construct the graph representation of the text, update the node representations using the GNN model, and predict the relations based on the relation-specific parameters. Extract the predicted relations between entities from the graph representation.

Final outcome





Graph from transformer model using Graphviz



Chapter 5

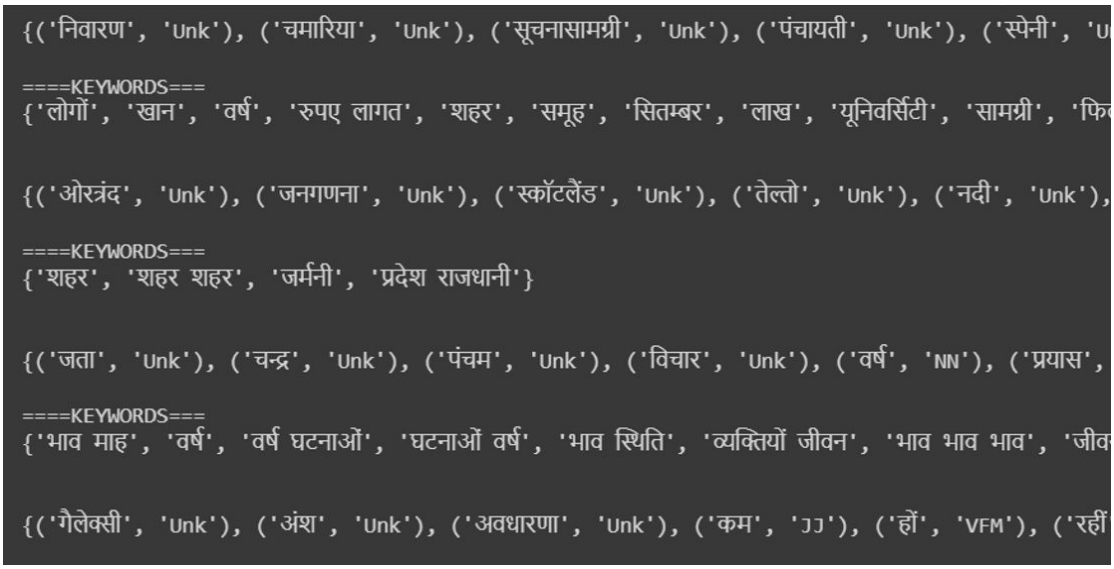
RESULTS AND DISCUSSION

Sentence segmentation

The sentences are segmented into words when a punctuation is encountered.

Tokenization

The paragraphs and sentences are split into smaller units (words) that can be more easily assigned meaning. The keywords were generated. The stop words in Hindi were dropped and the punctuations were removed.



Parts of Speech Tagging Parts of Speech Tagging (PoS)

PoS tagging converts a word to a list of tuples (word, tag). Since it was a Hindi dataset, there were unknown tags. It was handled using google translate.

```
================================New Tagged words================================
[('विटीफीड', 'NN'), ('इंटरनेट', 'NN'), ('मीडिया', 'NN'), ('सूचनासामग्री', 'NN'), ('प्रदाता
================================New Tagged words================================
[('ज़ागान', 'NN'), ('पश्चिमी', 'JJ'), ('पोलैंड', 'NN'), ('शहर', 'NN'), ('शहर', 'NN'),
================================New Tagged words================================
[('ज्योतिष', 'NN'), ('शास्त्र', 'NN'), ('माध्यम', 'NN'), ('जीवन', 'NN'), ('बारीक', 'NN'
```

Translation of English

	Sentence id	Word	Transword
0	1	विटीफीड	vitfeed
1	1	इंटरनेट	Internet
2	1	मीडिया	Media
3	1	सूचनासामग्री	information material
4	1	प्रदाता	the provider
5	1	कम्पनी	Company
7	2	आरम्भ	start
8	2	2014	2014
10	3	इन्दौर	Indore

Named Entity Recognition (NER) Begin, Inside, Outside (BIO) Tagging

Each token is tagged to one of the seven standard NLP BIO – Beginner, Inside, Outside - tags (I - Location, B - Organization, I - Person, O, B - Person, I - Organization, B

– Location). Using the BIO tags, numeric feature is extracted. It is then converted to numeric feature vectors.

	Sentence id	Word	Transword	Tag
0	1	विटीफीड	vitfeed	O
1	1	इंटरनेट	Internet	O
2	1	मीडिया	Media	O
3	1	सूचनासामग्री	information material	O
4	1	प्रदाता	the provider	O
5	1	कम्पनी	Company	B-ORGANIZATION
7	2	आरम्भ	start	O
8	2	2014	2014	O

Numeric Feature Extraction

['B-LOCATION', 'I-LOCATION', 'I-ORGANIZATION', 'B-ORGANIZATION', 'B-PERSON', 'O', 'I-PERSON']

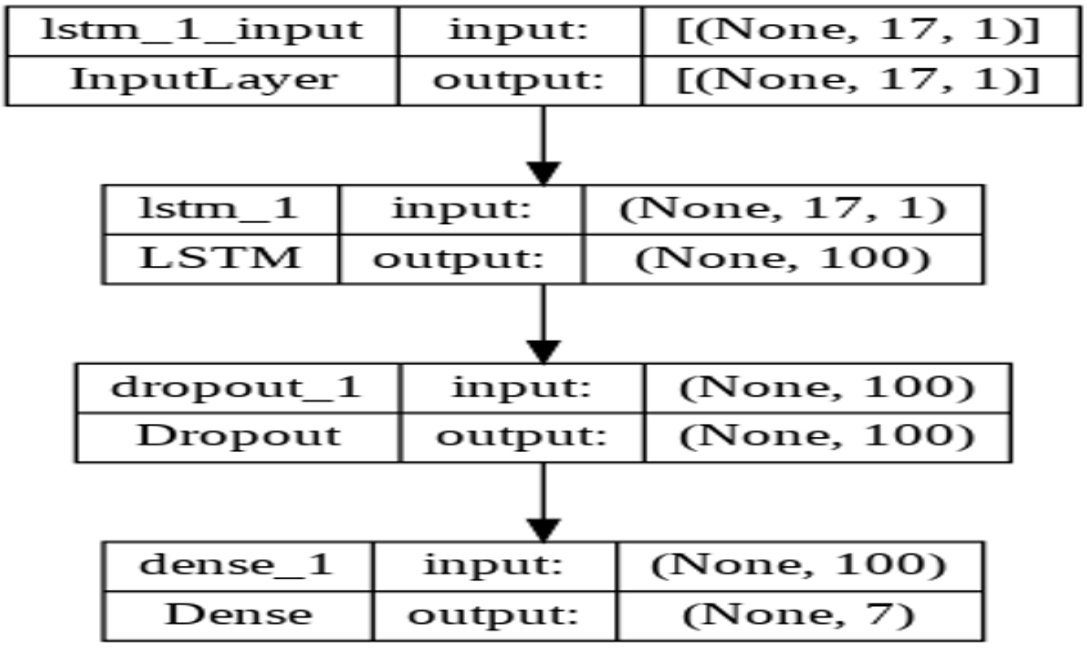
	bias	word	word[-3:]	word[-2:]	word.isdigit()	word.isletter()	word.isAlpha()	word.Tag	BOS	+1:word	+1:word.isdigit()	+1:word.isletter()	+1:word.isAlpha()	-1:word	-1:word.
0	1.0	122	519	519	0	1	1	5	1.0	418.0	0.0	1.0	1.0	NaN	
1	1.0	418	418	418	0	1	1	5	NaN	602.0	0.0	1.0	1.0	122.0	
2	1.0	602	602	602	0	1	1	5	NaN	30.0	0.0	1.0	1.0	418.0	
3	1.0	30	376	455	0	1	1	5	NaN	45.0	0.0	1.0	1.0	602.0	
4	1.0	45	435	518	0	1	1	5	NaN	233.0	0.0	1.0	1.0	30.0	
5	1.0	233	505	548	0	1	1	3	NaN	NaN	NaN	NaN	NaN	45.0	
6	1.0	150	150	150	0	1	1	5	1.0	454.0	1.0	0.0	1.0	NaN	
7	1.0	454	454	454	1	0	1	5	NaN	NaN	NaN	NaN	NaN	150.0	
8	1.0	467	467	467	0	1	1	5	1.0	56.0	0.0	1.0	1.0	NaN	
9	1.0	56	56	56	0	1	1	5	NaN	NaN	NaN	NaN	NaN	467.0	

Bidirectional Long Short-Term Memory (Bi-LSTM) for Named Entity Recognition (NER)

Accuracy and validation accuracy are increasing as the epoch number increases. This

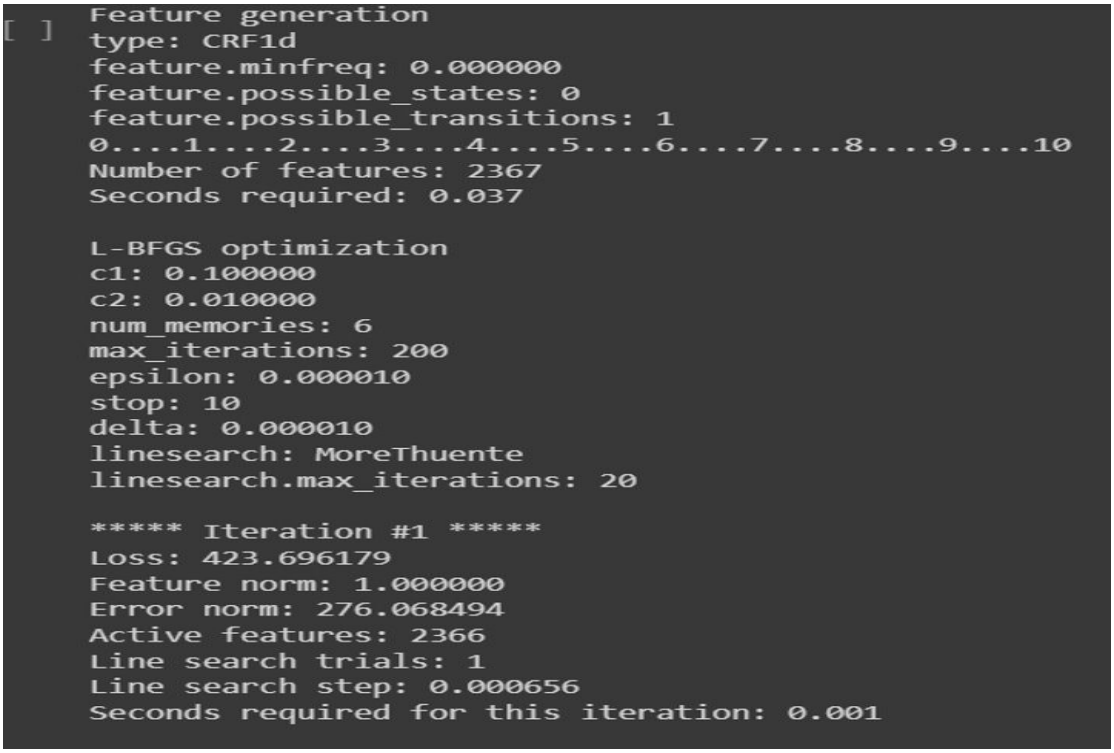
shows that the model is working well. Loss and validation loss are decreasing from 1.7627 to 1.2389.

```
Epoch 1/5
24/24 [==============================] - 4s 54ms/step - loss: 0.8039 - accuracy: 0.8284 - val_loss: 0.4506 - val_accuracy: 0.8978
Epoch 2/5
24/24 [==============================] - 0s 18ms/step - loss: 0.2630 - accuracy: 0.9270 - val_loss: 0.2076 - val_accuracy: 0.8978
Epoch 3/5
24/24 [==============================] - 0s 19ms/step - loss: 0.1067 - accuracy: 0.9608 - val_loss: 0.0612 - val_accuracy: 1.0000
Epoch 4/5
24/24 [==============================] - 0s 18ms/step - loss: 0.0372 - accuracy: 0.9878 - val_loss: 0.0119 - val_accuracy: 1.0000
Epoch 5/5
24/24 [==============================] - 0s 19ms/step - loss: 0.0611 - accuracy: 0.9824 - val_loss: 0.0187 - val_accuracy: 1.0000
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm_1 (LSTM)               (None, 100)               40800

 dropout_1 (Dropout)         (None, 100)               0

 dense_1 (Dense)             (None, 7)                 707

=================================================================
Total params: 41,507
Trainable params: 41,507
Non-trainable params: 0
```



Conditional Random Field

In Conditional Random Field (CRF), the prediction is modeled as a sequence labeling problem, where the goal is to assign labels to each element in a sequence based on the observed input features. The model considers the dependencies among the labels and utilizes contextual information to make predictions.



Relation Extraction

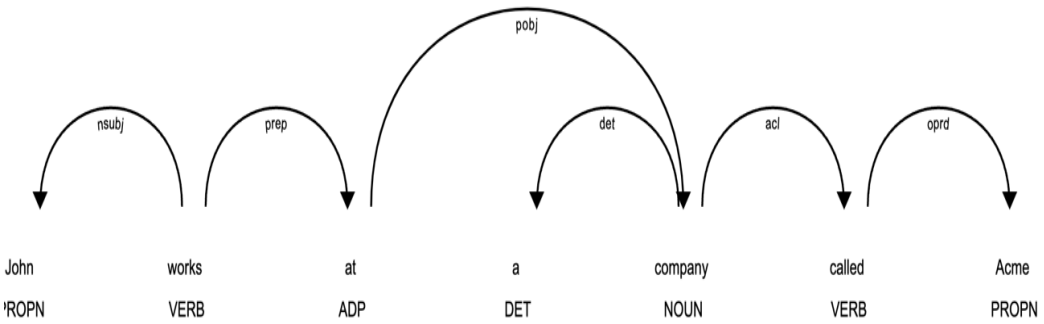
Extracting relations from a sample using the nlp pipeline. Each sentence is processed individually, and relations are extracted using the relations attribute. The extracted relations are then printed or processed.

Spacy library for extracting relation

It performs dependency parsing, which is a technique used to analyze and represent the

grammatical relationships between words in a sentence.

```
John nsubj loves VERB []
loves ROOT loves VERB [John, Mary, .]
Mary dobj loves VERB []
. punct loves VERB []
```



F-Measure or F-Score (F1)

F1 score is the harmonic mean of precision and recall and provides a balanced measure of the model's performance. It combines both precision and recall into a single metric, giving equal weight to both.

```
Test Results

Prediction time:0:00:36.638012
Copy On
Overall F1
(0.763, 0.602, 0.673)
Copy Off
Prediction time:0:00:33.451585
Overall F1
(0.678, 0.545, 0.604)
```

Chapter 6

CONCLUSION

This project focuses on advancing natural language processing and information extraction for the Hindi language through the application of relational extraction techniques and the Generated parameter Graph Neural Network Generated parameter Graph Neural Network (GPGNN) algorithm. By leveraging these techniques, the project aims to automatically identify and extract structured information and relationships from unstructured or semi-structured Hindi language.

The project encompasses several essential steps to achieve its objectives. The text undergoes pre-processing to identify and extract entities of interest, such as people, organizations, locations, and other relevant entities. Techniques like named entity recognition (NER) or part-of-speech tagging are employed to accurately classify and categorize these entities.

Once the entities are identified, the GPGNN algorithm is utilized to analyzes the text, considering contextual information, syntactic dependencies, to extract meaningful relationships between the recognized entities. By combining graph-based representations and neural networks, the GPGNN algorithm effectively captures the semantic connections and interactions within the Hindi text.

To enhance understanding and accessibility, the project focuses on constructing a knowledge graph. This graph-based representation organizes the extracted entities and relationships in a structured format, providing a comprehensive view of the interconnectedness between them. The knowledge graph facilitates in-depth analysis, empowering researchers and practitioners to perform complex queries, and uncover hidden patterns within the Hindi text.Throughout the project, evaluation metrics such as precision, recall, and F1-score are employed to assess the accuracy of the extracted relationships and the overall performance of the GPGNN algorithm.

Additionally, the project implements a user interface using Flask, a popular Python web framework. Flask provides a flexible and effective platform for creating web apps. The Flask UI allows users to input Hindi text, initiate the information extraction process, and visualize the extracted knowledge graph.It's simplicity make it well-suited for this project's requirement.It allows for easy integration with the backend algorithms and data processing pipelines, ensuring efficient communication between the user interface and the underlying NLP components. It facilitates the presentation of extracted information, including the identified entities, relationships, and the constructed knowledge graph.

Through the construction of a knowledge graph and the implementation of Flask as the user interface framework, the project enhances the usability and accessibility of the developed system. The knowledge graph organizes the extracted information in a structured format. The Flask UI provides a user-friendly platform for users to interact and visualize the extracted knowledge graph.

The successful completion of this project has significant practical implications. It advances NLP and information extraction, enabling improved text understanding, information retrieval, and data analytics. The extracted structured information and relationships can be utilized in various applications, including domain-specific knowledge extraction, content recommendation systems, and intelligent decision-making systems.

In summary, this project showcases the effectiveness of GP-GNN and training NER models using SpaCy's prebuilt transformer models for enhancing sentence understanding and extracting named entities from Hindi text. The utilization of GNN and graph properties enables the capture of relationships and enrichment of semantic information. Fine-tuning prebuilt transformer models for NER in Hindi, along with dependency parsing, further enhances the extraction of meaningful information.

Chapter 7

FUTURE ENHANCEMENT

1. Model Optimization : Explore various techniques for improving the performance of the GPGNN algorithm and NER models. This can involve using extra datasets relevant to Hindi to fine-tune the prebuilt transformer models, experimenting with various hyperparameters, or applying sophisticated optimization approaches.

2. Semi-Supervised or Unsupervised Learning : Explore techniques that leverage partially or inaccurately labeled data to improve the effectiveness of NER models and the GPGNN algorithm. These techniques fall under the categories of semi-supervised or unsupervised learning. Potential approaches to consider include self-training, active learning, and unsupervised pretraining, which can enable the utilization of additional data and enhance the overall performance of the system.

3. Handling Ambiguity and Polysemy : Address the difficulties that the Hindi language faces as a result of ambiguity and polysemy. Create methods for accurately resolving entity ambiguities and resolving conflicting relationship interpretations. This may entail using contextual data, word embeddings, or other Hindi-specific linguistic resources.

4. Incremental Learning and Adaptation : Design the system to adapt and learn continuously from user interactions and feedback for incremental learning. Include tools that let the system iteratively update and improve its knowledge graph and models depending on fresh information or user annotations. By doing this, the system is kept current with changing linguistic trends and user demands.

5. Cross-Lingual and Multilingual Support : Extend the techniques to handle multiple languages, enabling cross-lingual or multilingual information extraction and analysis. This can involve developing techniques for aligning and transferring knowledge

across languages or integrating multilingual resources into the system.

6. Real-Time Processing and Scalability : Optimize the techniques to handle large-scale data and enable real-time processing. This can involve implementing parallel processing, distributed computing, or utilizing specialized hardware to enhance the system's scalability and efficiency.

7. User Interaction and Feedback Mechanisms : To enhance the functionality of the system, incorporate user input and engagement elements. Allow users to add annotations, corrections, or domain-specific knowledge that can be utilized to improve the knowledge graph and the extraction models.

8. Error Analysis and Interpretability : To comprehend the limitations and sources of faults in the procedures, perform a thorough error analysis. Create techniques for analyzing and explaining information extraction outcomes, giving the system's decision-making process insights and improving transparency.

REFERENCES

1. Carbonell, M., Riba, P., Villegas, M., Fornés, A., and Lladós, J. (2021). "Named entity recognition and relation extraction with graph neural networks in semi structured documents." *2020 25th International Conference on Pattern Recognition (ICPR)*, IEEE. 9622–9627.

2. Cetoli, A., Bragaglia, S., O'Harney, A. D., and Sloan, M. (2017). "Graph convolutional networks for named entity recognition." *arXiv preprint arXiv:1709.10053*.

3. Chiu, J. P. and Nichols, E. (2016). "Named entity recognition with bidirectional lstm-cnns." *Transactions of the association for computational linguistics*, 4, 357–370.

4. Dessì, D., Osborne, F., Recupero, D. R., Buscaldi, D., and Motta, E. (2021). "Generating knowledge graphs by employing natural language processing and machine learning techniques within the scholarly domain." *Future Generation Computer Systems*, 116, 253–264.

5. Shao, Y., Lin, C.-W., Srivastava, G., Jolfaei, A., Guo, D., and Hu, Y. (2021a). "Self-attention-based conditional random fields latent variables model for sequence labeling." *Pattern Recognition Letters*, 145.

6. Shao, Y., Lin, J. C.-W., Srivastava, G., Jolfaei, A., Guo, D., and Hu, Y. (2021b). "Self-attention-based conditional random fields latent variables model for sequence labeling." *Pattern Recognition Letters*, 145, 157–164.

7. Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., and Philip, S. Y. (2020). "A comprehensive survey on graph neural networks." *IEEE transactions on neural networks and learning systems*, 32(1), 4–24.

8. Zeng, D., Liu, K., Lai, S., Zhou, G., and Zhao, J. (2014). "Relation classification via convolutional deep neural network." *Proceedings of COLING 2014, the 25th international conference on computational linguistics: technical papers*. 2335–2344.

9. Zeng, W., Lin, Y., Liu, Z., and Sun, M. (2016). "Incorporating relation paths in neural relation extraction." *arXiv preprint arXiv:1609.07479*.